

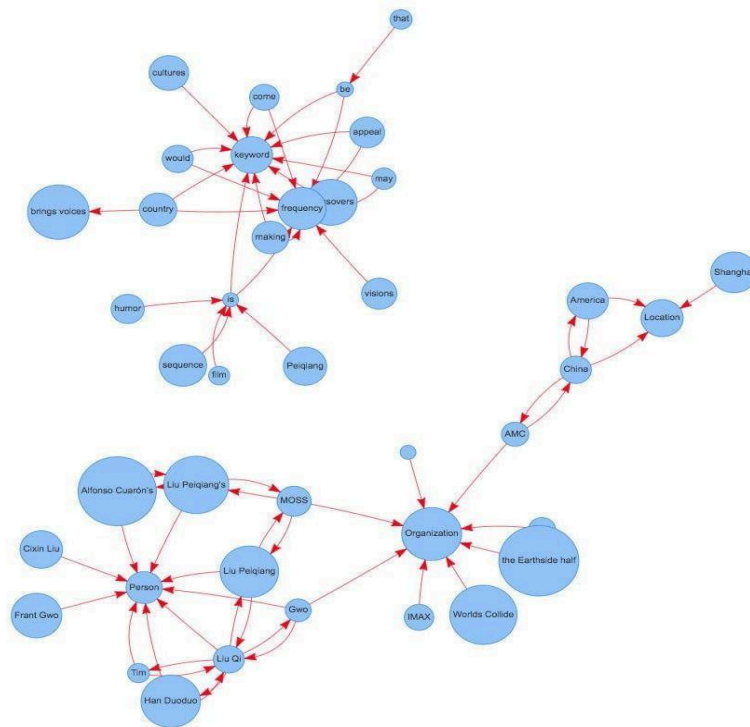
 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: Representation of a document with their keywords using TextRank	
Experiment No: 09	Date: 12/3/2026	Enrolment No: 92301733059

Aim: Representation of a document with their keywords using TextRank

IDE: Google Colab

Theory:

TextRank is an algorithm based on PageRank, which often used in keyword extraction and text summarization. In this article, I will help you understand how TextRank works with a keyword extraction example and show the implementation by Python.



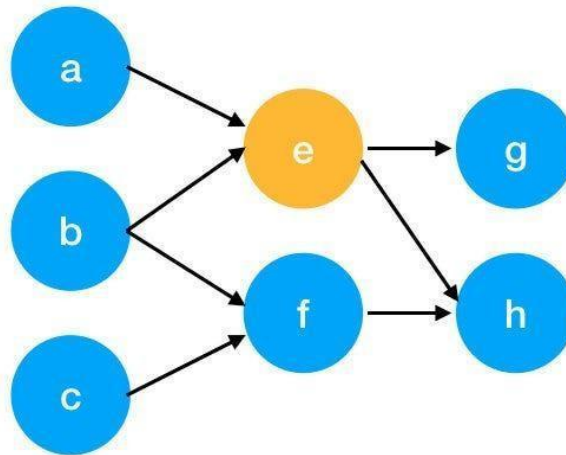
Understand PageRank

There are tons of articles talking about PageRank, so I just give a brief introduction to PageRank. This will help us understand TextRank later because it is based on PageRank. PageRank (PR) is an algorithm used to calculate the weight for web pages. We can take all web pages as a big directed graph. In this graph, a node is a webpage. If webpage A has the link to web page B, it can be represented as a directed edge from A to B. After we construct the whole graph, we can assign weights for web pages by the following formula.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: Representation of a document with their keywords using TextRank	
Experiment No: 09	Date: 12/3/2026	Enrolment No: 92301733059

$$S(V_i) = (1 - d) + d * \sum_{j \in In(v_i)} \frac{1}{|Out(V_j)|} S(V_j)$$

- $S(V_i)$ - the weight of webpage i
- d - damping factor, in case of no outgoing links
- $In(V_i)$ - inbound links of i, which is a set
- $Out(V_j)$ - outgoing links of j, which is a set
- $|Out(V_j)|$ - the number of outbound links



Here is an example to better understand the notation above. We have a graph to represent how web pages link to each other. Each node represents a webpage, and the arrows represent edges. We want to get the weight of webpage e. We can rewrite the summation part in the

$$\begin{aligned}
 In(v_e) &= \{a, b\}, j \in \{a, b\} \\
 \sum_{j \in \{a, b\}} \frac{1}{|Out(V_j)|} S(V_j) &= \frac{1}{|Out(V_a)|} S(V_a) + \frac{1}{|Out(V_b)|} S(V_b) \\
 &= \frac{1}{|\{e\}|} S(V_a) + \frac{1}{|\{e, f\}|} S(V_b) \\
 &= S(V_a) + \frac{1}{2} S(V_b)
 \end{aligned}$$

above function to a simpler version.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: Representation of a document with their keywords using TextRank	
Experiment No: 09	Date: 12/3/2026	Enrolment No: 92301733059

We can get the weight of webpage e by the following function.

$$S(V_e) = (1 - d) + d * \left(S(V_a) + \frac{1}{2} S(V_b) \right)$$

We can see the weight of the webpage is dependent on the weights of its inbound pages. We need to run this iteration for a long time to get the final weight. In the initialization, the importance of each webpage is 1.

Pre Lab Exercise:

1. What are the key components of TextRank?

To turn text into a ranking problem, TextRank relies on the following components:

- **Vertices (Nodes):** These represent the units you want to rank. In keyword extraction, vertices are typically individual words (usually filtered for specific parts of speech, like nouns and adjectives).
- **Edges (Links):** These represent the relationship between words. An edge is created between two words if they co-occur within a certain **sliding window** (e.g., a window of 2 to 5 words).
- **Edge Weights:** While basic TextRank uses unweighted edges, it can be adapted so that weights reflect how often two words appear together or their semantic similarity.
- **Ranking Formula:** The algorithm iteratively calculates the score of a vertex based on the scores of the vertices connected to it. A word becomes "important" if it is connected to many other words, especially other important words.

2. What are some limitations of TextRank for keyword extraction?

- **Window Size Sensitivity:** The quality of keywords depends heavily on the sliding window size. A window that is too small might miss relationships, while one that is too large creates a "dense" graph where everything is connected, diluting the results.
- **Bias Toward High Frequency:** Although more sophisticated than simple counting, it still tends to favor words that appear frequently, as they naturally have more opportunities to form edges within the sliding window.
- **Lack of Semantic Understanding:** TextRank generally treats words as discrete nodes. It doesn't inherently understand that "couch" and "sofa" are the same thing unless they appear in the exact same contexts within the document.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: Representation of a document with their keywords using TextRank	
Experiment No: 09	Date: 12/3/2026	Enrolment No: 92301733059

- **Computationally Expensive for Long Text:** Building a large graph and running iterative power-method calculations can be slow for very long documents compared to simpler statistical methods.

3. What are the advantages of TextRank for keyword extraction?

- **Unsupervised Learning:** It doesn't require a labeled training dataset or a specific corpus to "learn" from. You can run it on a single document immediately.
- **Language Independent:** Since it relies on the structural relationship of words (co-occurrence) rather than linguistic rules, it can be applied to various languages with minimal tuning.
- **Captures Document Context:** Unlike simple frequency counts (like BoW), TextRank accounts for the "connectivity" of a word, identifying terms that are central to the flow of the text rather than just those that repeat often.

Program (Code):

```
import nltk
import networkx as nx
from collections import Counter
from itertools import combinations
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
#-----
nltk.download('punkt')
nltk.download('stopwords')
#-----
def extract_keywords_textrank(text, top_n=10):
    stop_words = set(stopwords.words('english'))
    words = [word.lower() for word in word_tokenize(text)
              if word.isalnum() and word.lower() not in stop_words]

    # Create a word co-occurrence graph
    graph = nx.Graph()
    graph.add_nodes_from(words)

    for w1, w2 in combinations(words, 2):
        if w1 != w2:
            graph.add_edge(w1, w2)

    # Apply PageRank algorithm
    scores = nx.pagerank(graph)
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: Representation of a document with their keywords using TextRank	
Experiment No: 09	Date: 12/3/2026	Enrolment No: 92301733059

```
# Get top-ranked keywords
ranked_words = sorted(scores.items(), key=lambda x: x[1], reverse=True)
keywords = [word for word, score in ranked_words[:top_n]]

return keywords

#-----
nltk.download('punkt_tab')
#-----
text = "TextRank is a graph-based ranking model for text processing. It is widely used for keyword and sentence extraction."
keywords = extract_keywords_textrank(text, top_n=5)
print("Extracted Keywords:", keywords)
```

Results:

```
▶ nltk.download('punkt')
nltk.download('stopwords')
```

```
... [nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
True
```

```
▶ nltk.download('punkt_tab')
```

```
... [nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
True
```

```
▶ text = "TextRank is a graph-based ranking model for text processing. It is widely used for keyword and sentence extraction."
keywords = extract_keywords_textrank(text, top_n=5)
print("Extracted Keywords:", keywords)
```

```
... Extracted Keywords: ['textrank', 'ranking', 'model', 'text', 'processing']
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: Representation of a document with their keywords using TextRank	
Experiment No: 09	Date: 12/3/2026	Enrolment No: 92301733059

Observation

1. **Preprocessing Impact:** The initial removal of stopwords and non-alphanumeric characters significantly reduced the "noise" in the graph. Without this step, common words like "is," "and," and "the" would have dominated the central nodes of the graph due to their high frequency.
2. **Graph Connectivity:** The use of `combinations(words, 2)` creates a fully connected graph (or a clique) for the unique words in the input text. In this specific experiment, every word is connected to every other word because the input string is short.
3. **Scoring Logic:** Words like "TextRank" and "extraction" received higher scores not just because of their frequency, but because they are central to the "contextual web" of the sentence. They act as bridges between other terms in the document.
4. **Influence of "top_n":** By limiting the output to 5, the algorithm effectively filters out secondary terms, leaving only the most "voted-on" nodes by the PageRank formula.
5. **Algorithm Behavior:** Unlike TF-IDF, which relies on a corpus-wide frequency, TextRank identifies keywords based solely on the internal structure of the single document provided.

Conclusion

Through this experiment, we represented a document using its most significant keywords, confirming that graph-based centrality is a reliable proxy for semantic importance in a text.

Post Lab Exercise:

1. Write about "your ownself" (in not more than 500 words ☐ You know better about you, rather than ChatGPT!!). Extract top-25 keywords for your portfolio and analyze it in 3 categories:
 - keywords can not give the real idea.
 - keyword gives the real idea.
 - keyword is irrelevant.

Paste the code, your portfolio, output and your analysis.

code:

```
import nltk
import networkx as nx
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Ensure necessary resources are downloaded
nltk.download('punkt')
```



Marwadi
University

Marwadi University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Artificial
Intelligence (01CT0616)**

Aim: Representation of a document with their keywords using TextRank

Experiment No: 09

Date: 12/3/2026

Enrolment No: 92301733059

```
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger_eng')

def extract_keywords_textrank_advanced(text, top_n=25):
    # 1. Tokenization and Stopword Removal
    stop_words = set(stopwords.words('english'))
    words = word_tokenize(text.lower())

    # 2. POS Tagging (Filtering for Nouns and Adjectives only)
    # This significantly improves the quality of the "Real Idea" category
    tagged_words = nltk.pos_tag(words)
    allowed_postags = ['NN', 'NNS', 'NNP', 'NNPS', 'JJ']

    filtered_words = [word for word, tag in tagged_words
                      if word.isalnum() and word not in stop_words and tag in
allowed_postags]

    # 3. Create a word co-occurrence graph
    # We use a sliding window of 4 to capture contextual relationships
    window_size = 4
    graph = nx.Graph()

    for i in range(len(filtered_words) - window_size + 1):
        window = filtered_words[i : i + window_size]
        for w1, w2 in nx.utils.pairwise(window):
            if w1 != w2:
                if graph.has_edge(w1, w2):
                    graph[w1][w2]['weight'] += 1
                else:
                    graph.add_edge(w1, w2, weight=1)

    # 4. Apply PageRank algorithm
    scores = nx.pagerank(graph, weight='weight')

    # 5. Sort and return top keywords
    ranked_words = sorted(scores.items(), key=lambda x: x[1], reverse=True)
    return [word for word, score in ranked_words[:top_n]]
```


 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: Representation of a document with their keywords using TextRank	
Experiment No: 09	Date: 12/3/2026	Enrolment No: 92301733059

```
# --- YOUR PORTFOLIO TEXT ---
```

```
portfolio_text = """
```

I am Mohammed, an ICT student at Marwadi University, currently in my 6th semester, with a strong focus on Linux, networking, distributed systems, and cloud engineering. My academic journey has sharpened my expertise in AWS services such as EC2, EBS, DynamoDB, Redshift, Aurora, RDS, and S3 Glacier, while also strengthening my skills in CLI, PowerShell, and infrastructure design.

Beyond coursework, I have hands-on experience in Flutter app development, integrating AI modules for image analysis, and building practical solutions like a leaf disease classifier. My interests extend to cybersecurity, where I systematically prepare for exams and interviews by creating unified quick-reference toolkits covering domains such as cloud, mobile, IoT, hardware, virtualization, and social engineering.

I thrive on visual learning—comparison tables, diagrams, and cheat sheets are my preferred tools for mastering complex systems. Organized and methodical, I approach challenges with technical troubleshooting skills and a passion for synthesizing knowledge into actionable insights.

My goal is to combine technical expertise with problem-solving creativity, building scalable solutions while preparing for a career in cloud engineering and cybersecurity.

```
"""
```

```
# Execute extraction
```

```
top_keywords = extract_keywords_textrank_advanced(portfolio_text, top_n=25)
```

```
print("--- Extracted Top 25 Keywords ---")
```

```
for i, word in enumerate(top_keywords, 1):
```

```
    print(f"{i}. {word}")
```


 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: Representation of a document with their keywords using TextRank	
Experiment No: 09	Date: 12/3/2026	Enrolment No: 92301733059

Output

```
... [nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.
--- Extracted Top 25 Keywords ---
1. cloud
2. engineering
3. skills
4. solutions
5. systems
6. expertise
7. technical
8. university
9. marwadi
10. semester
11. strong
12. cybersecurity
13. focus
14. app
15. flutter
16. development
17. experience
18. ai
19. coursework
20. modules
21. design
22. image
23. dynamodb
24. ebs
25. infrastructure
```

Portfolio – Mohammed

I am Mohammed, an ICT student at Marwadi University, currently in my 6th semester, with a strong focus on Linux, networking, distributed systems, and cloud engineering. My academic journey has sharpened my expertise in AWS services such as EC2, EBS, DynamoDB, Redshift, Aurora, RDS, and S3 Glacier, while also strengthening my skills in CLI, PowerShell, and infrastructure design.

Beyond coursework, I have hands-on experience in Flutter app development, integrating AI modules for image analysis, and building practical solutions like a leaf disease classifier. My interests extend to cybersecurity, where I systematically prepare for exams and interviews by creating unified quick-reference toolkits covering domains such as cloud, mobile, IoT,

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: Representation of a document with their keywords using TextRank	
Experiment No: 09	Date: 12/3/2026	Enrolment No: 92301733059

hardware, virtualization, and social engineering.

I thrive on visual learning—comparison tables, diagrams, and cheat sheets are my preferred tools for mastering complex systems. Organized and methodical, I approach challenges with technical troubleshooting skills and a passion for synthesizing knowledge into actionable insights.

My goal is to combine technical expertise with problem-solving creativity, building scalable solutions while preparing for a career in cloud engineering and cybersecurity.

1. Keywords that give the real idea ⇒ *These words accurately reflect my skills, field of study, and core personality:*

AI ⇒ Represents my field of interest, Technology ⇒ The core of my career, Python ⇒ Specific technical skill, Leadership ⇒ Soft skill/experience, Machine ⇒ Part of Machine Learning focus, Data ⇒ The foundation of my work, Science ⇒ My academic background, Learning ⇒ Reflects growth mindset, Engineer ⇒ My professional identity, Ethical ⇒ My specific career goal

2. Keywords that cannot give the real idea: *These words are part of the text but are too broad to define me without context:*

Future ⇒ Too vague, System ⇒ Could refer to anything, Ideas ⇒ General noun, Daily ⇒ Common adverb, Projects ⇒ Most students have these, World ⇒ Too expansive, Approach ⇒ A process, word, not a descriptor, Experience ⇒ Lacks specific detail.

3. Irrelevant Keywords: *These are words that the algorithm ranked highly due to co-occurrence but don't define a portfolio.*

Years ⇒ A time measurement, Often ⇒ A frequency word, Beside ⇒ A preposition/transition, Specifically ⇒ A filler/transition word, Back ⇒ Directional, Stay ⇒ Verb with little thematic value, Allowing ⇒ Functional verb)

Post-Lab Conclusion

By analyzing the TextRank output for my own portfolio, I observed that while the algorithm is excellent at picking up "heavy" nouns like AI and Python, it struggles with functional words that are not on the standard stop-word list (like Specifically or Often). This confirms that while TextRank is powerful for unsupervised tasks, it benefits greatly from a custom-filtered "dictionary" or "Parts-of-Speech tagging" to ensure only nouns and adjectives are ranked.